

6: Linear Regression

Date: February 3, 2026

We've now developed a general optimization-based view of learning:

1. First, pick a hypothesis class $\mathcal{H}$ that contains models h parameterized by some set of parameters $\mathbf{w}$.
2. Next, pick a *loss function* $\ell(\hat{y}, y) = \ell(h(\mathbf{x}), y)$ that measures how bad the prediction $\hat{y} = h(\mathbf{x})$ a model makes is.
3. Finally, we use gradient descent to minimize the average loss across a training dataset $\mathcal{D} = \{(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_n, y_n)\}$. This fixes a measure of empirical risk, which is the average loss over a training dataset, $\min_{h \in \mathcal{H}} \hat{R}(h) = \frac{1}{n} \sum_{i=1}^n \ell(h(\mathbf{x}_i), y_i)$.

Looking above at the steps, there are really only two choices we need to make to define the kind of models we're learning: (1) what *hypothesis class* $\mathcal{H}$ should we use, and (2) what *loss function* should we use? Today, we'll make these choices concrete by studying our first regression algorithm: *linear regression*.

1 Linear Regression

Concretely, the ingredients for linear regression are:

- **Input space.** Input space $\mathcal{X} = \mathbb{R}^d$ are real-valued feature vectors.
- **Label space.** Label space $\mathcal{Y} = \mathbb{R}$ are real-valued labels.
- **Hypothesis class.** We will use *linear models*:

$$\mathcal{H} := \left\{ \mathbf{x} \mapsto \mathbf{w}^\top \mathbf{x} + b \mid \mathbf{w} \in \mathbb{R}^d, b \in \mathbb{R} \right\}. \quad (1)$$

We directly use $h(\mathbf{x})$ as a real-valued prediction. For notational convenience, we will often *absorb the bias* by augmenting each feature vector with a constant 1 and folding b into $\mathbf{w}$; after this augmentation, we will write $h(\mathbf{x}) = \mathbf{w}^\top \mathbf{x}$.

- **Loss function.** We will use the squared or ℓ_2 -loss:

$$\ell(\hat{y}, y) = (\hat{y} - y)^2. \quad (2)$$

In this setup, the empirical risk concretely becomes:

$$\hat{R}(\mathbf{w}) = \frac{1}{n} \sum_{i=1}^n (\mathbf{w}^\top \mathbf{x}_i - y_i)^2 \quad (3)$$

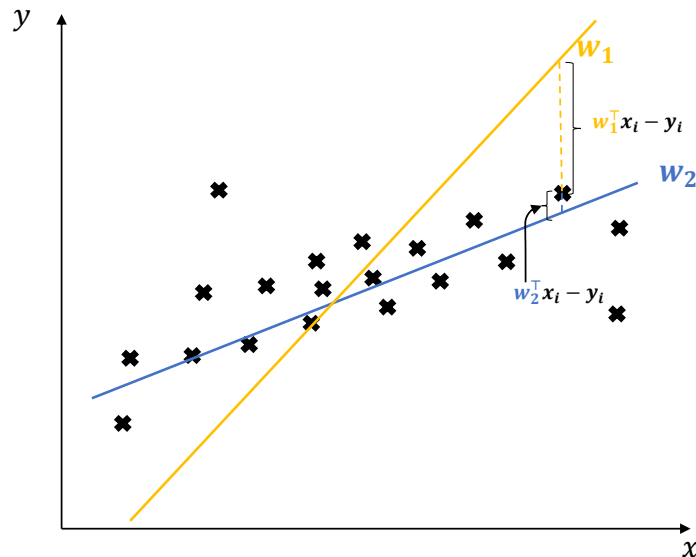


Figure 1: Two example potential linear predictors for some sample data, whose parameters would be given by $\mathbf{w}_1$ and $\mathbf{w}_2$. The orange and blue dotted lines measure the distance between the prediction made by the line and the actual label observed for a particular input $\mathbf{x}_i$. The empirical risk is to take the average squared length of these dotted lines across all $\mathbf{x}_i$. The loss for $\mathbf{w}_1$ will be higher, because these dotted lines will be much longer on average.

and the optimal linear regression model is just given by $\mathbf{w}^* = \arg \min_{\mathbf{w}} \hat{R}(\mathbf{w})$. A graphical intuition for this objective is in Figure 1.

It will generally be very helpful to write this down in matrix form. We can represent the data as a matrix $\mathbf{X}$ and vector $\mathbf{y}$:

$$\mathbf{X} = \begin{bmatrix} - & \mathbf{x}_1^\top & - \\ - & \mathbf{x}_2^\top & - \\ & \vdots & \\ - & \mathbf{x}_n^\top & - \end{bmatrix}, \quad \mathbf{y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}$$

Here, $\mathbf{X}$ is an $n \times d$ matrix, where the i th row of the matrix holds the feature vector for the i th training datapoint, $\mathbf{x}_i$. To give you a better flavor of how to think with matrices—which will become relevant when you inevitably implement this in PyTorch—let’s build up the loss in Equation 3 piece by piece.

Step 1: Get predictions. The first thing we’ll do in matrix form is get the predictions of the current model (e.g., given a current weight vector $\mathbf{w}$) on each data point $\mathbf{x}_i$. To do this, observe that $\mathbf{X}\mathbf{w}$ takes each row of $\mathbf{X}$ and computes a dot product of that row with $\mathbf{w}$. This is just definitionally what a matrix vector product does. In other words:

$$\mathbf{X}\mathbf{w} = \begin{bmatrix} \mathbf{w}^\top \mathbf{x}_1 \\ \mathbf{w}^\top \mathbf{x}_2 \\ \vdots \\ \mathbf{w}^\top \mathbf{x}_n \end{bmatrix}$$

This is looking pretty good! The above computation lets us get all of the predictions without for loops.

Step 2: Compute differences from labels. This one is pretty easy: if we compute $\mathbf{X}\mathbf{w} - \mathbf{y}$, that vector difference is just:

$$\mathbf{X}\mathbf{w} = \begin{bmatrix} \mathbf{w}^\top \mathbf{x}_1 - y_1 \\ \mathbf{w}^\top \mathbf{x}_2 - y_2 \\ \vdots \\ \mathbf{w}^\top \mathbf{x}_n - y_n \end{bmatrix}$$

which looks exactly like the innermost portion of the empirical risk above (Equation 3).

Step 3: Compute the loss. To finish computing the loss, we need to do two things. First, we need to square each entry in the vector above, $\mathbf{X}\mathbf{w} - \mathbf{y}$. Second, we need to sum all of the entries together. One way to accomplish this would definitely be to simply apply `.pow(2).sum()` to the vector in code. However, observe that for a vector $\mathbf{a} = [a_1, \dots, a_n]$, $\|\mathbf{a}\|_2^2 = \mathbf{a}^\top \mathbf{a} = a_1^2 + a_2^2 + \dots + a_n^2$, which is exactly the operation we want to perform here. Thus, our loss function can be computed in matrix form as:

$$\hat{R}(\mathbf{w}) = \|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2 = (\mathbf{X}\mathbf{w} - \mathbf{y})^\top (\mathbf{X}\mathbf{w} - \mathbf{y}) \quad (4)$$

2 Optimizing linear regression

The gradient of the non matrix form loss in Equation 3 is:

$$\nabla_{\mathbf{w}} \hat{R}(\mathbf{w}) = \frac{2}{n} \sum_{i=1}^n (\mathbf{w}^\top \mathbf{x}_i - y_i) \mathbf{x}_i \quad (5)$$

To derive this, we apply three rules from calculus.

1. The derivative of the sum of a bunch of functions equals the sum of the derivatives, i.e.

$$\nabla_{\mathbf{w}} \hat{R}(\mathbf{w}) = \frac{1}{n} \sum_{i=1}^n \nabla_{\mathbf{w}} \left((\mathbf{w}^\top \mathbf{x}_i - y_i)^2 \right)$$

2. Apply the chain rule to the squared term, i.e.

$$\nabla_{\mathbf{w}} \hat{R}(\mathbf{w}) = \frac{2}{n} \sum_{i=1}^n (\mathbf{w}^\top \mathbf{x}_i - y_i) \nabla_{\mathbf{w}} \left(\mathbf{w}^\top \mathbf{x}_i - y_i \right)$$

3. Apply the gradient to the remaining terms, i.e.

$$\nabla_{\mathbf{w}} \hat{R}(\mathbf{w}) = \frac{2}{n} \sum_{i=1}^n (\mathbf{w}^\top \mathbf{x}_i - y_i) \mathbf{x}_i$$

Note that the linear regression loss is convex. In convex problems, we can find the global minimizer when the gradient is zero. In this case, we can use this property to find a closed form solution! Setting the gradient to zero and solving for $\mathbf{w}$ we get:

$$\begin{aligned}\nabla_{\mathbf{w}} \hat{R}(\mathbf{w}) &= \frac{2}{n} \sum_{i=1}^n (\mathbf{w}^\top \mathbf{x}_i - y_i) \mathbf{x}_i = 0 \\ \implies \left(\sum_{i=1}^n \mathbf{x}_i \mathbf{x}_i^\top \right) \mathbf{w} &= \sum_{i=1}^n y_i \mathbf{x}_i\end{aligned}$$

For convenience, it would be useful to view this in matrix form. Using $\mathbf{X}$ and $\mathbf{y}$ as defined before,

$$\left(\sum_{i=1}^n \mathbf{x}_i \mathbf{x}_i^\top \right) \mathbf{w} = \sum_{i=1}^n y_i \mathbf{x}_i \iff \mathbf{X}^\top \mathbf{X} \mathbf{w} = \mathbf{X}^\top \mathbf{y}.$$

These are called the normal equations for least squares regression. Now if $\mathbf{X}^\top \mathbf{X}$ is full rank (invertible) then we have,

$$\hat{\mathbf{w}} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}.$$

This gives us the prediction $\hat{\mathbf{y}} = \mathbf{X} \hat{\mathbf{w}} = \mathbf{X} (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$ which is the projection of $\mathbf{y}$ onto the subspace spanned by the columns of $\mathbf{X}$.

3 Can linear regression be too powerful?

Given the simple picture in Figure Equation 1, it may initially seem silly to even ask this question. In order to make the blue line parameterized by $\mathbf{w}_2$ seemingly fit the data well, we had to draw the data in a pretty straight line. It's not very hard to imagine the data looking just a little more complicated and there being *no* good linear fit.

Unfortunately, this is another example of where our intuition breaks down in high dimensions. Consider a scenario where the matrix $\mathbf{X}$ becomes invertible. To achieve this, we obviously need $n = d$, as only square matrices can be invertible in general. Furthermore, we need $\mathbf{X}$ to be nonsingular. In other words, we can't be able to write any data point $\mathbf{x}_i$ as a linear combination of other datapoints, $\sum_{j \neq i} \alpha_j \mathbf{x}_j$. Equivalently, $\mathbf{X} \in \mathbb{R}^{n \times n}$ and there is no vector $\mathbf{v} \neq 0$ so that $\mathbf{X} \mathbf{v} = 0$.

If $\mathbf{X}$ is invertible, what happens if we make predictions using $\mathbf{w}^*$? Recall that the matrix form of predictions is just $\mathbf{X} \mathbf{w}^*$. Plugging in our solution for $\mathbf{w}^*$ above:

$$\mathbf{X} \mathbf{w}^* = \mathbf{X} (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$$

Since $\mathbf{X}$ is invertible (and therefore $\mathbf{X}^\top$ must be as well), we can apply the identity $(AB)^{-1} = B^{-1}A^{-1}$ to $(\mathbf{X}^\top \mathbf{X})$ to get:

$$\mathbf{X} (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y} = \underbrace{\mathbf{X} \mathbf{X}^{-1}}_I \underbrace{\mathbf{X}^{-\top} \mathbf{X}^\top}_I \mathbf{y} = \mathbf{y}$$

Thus, in general if we have as many features as we have data points, unless we are in a very unlikely scenario where some data points are exactly linear combinations of others, we can achieve zero training error! Importantly, note that the above derivation assumed *nothing* about the labels

$\mathbf{y}$. In fact, the labels could have been completely random values, and linear regression would still be able to memorize the data. What if there are *more* features than data points? In this case, we end up solving a system of equations with more unknowns than equations, and there are *infinitely many* different linear functions $\mathbf{w}^\top \mathbf{x}$ that perfectly fit the training data.

While it may be surprising, this means that we actually do need to be careful with even simple linear models in the setting where we have many features, particularly if we start getting as many or nearly as many features as data points.

4 Regularization, Lasso, and Ridge Regression

Regularization is a way to augment an optimization problem like the one found in linear regression to make the learned model less powerful. To do this, we artificially *constrain* the solutions $\mathbf{w}^*$ that we find using a *regularization term* $t(\mathbf{w})$. A loose intuition is that $t(\mathbf{w})$ measures the “complexity” of a particular solution $\mathbf{w}$. There are two equivalent ways to add a complexity penalty to an empirical risk minimization problem:

$$\mathbf{w}^* = \arg \min_{\mathbf{w}} \hat{R}(\mathbf{w}) + \lambda t(\mathbf{w}) \iff \mathbf{w}^* = \arg \min_{\mathbf{w}} \hat{R}(\mathbf{w}) \text{ s.t. } t(\mathbf{w}) \leq B$$

On the left hand side, we simply add $\lambda t(\mathbf{w})$ to the original empirical risk directly. Intuitively, minimizing this essentially means we care about minimizing the complexity of our solution $\mathbf{w}$, and not just the loss. The *regularization parameter* λ controls how much we care about the loss versus the complexity. On the right side, we have the original minimization problem, but add a constraint that $t(\mathbf{w}) \leq B$. Here, we want to minimize the loss only, but we are saying that the solution $\mathbf{w}$ has a maximum complexity $t(\mathbf{w})$. Three important points:

1. These two formulations are entirely equivalent. For any λ , there exists a B so that the optimal solution $\mathbf{w}^*$ to the left optimization problem is the optimal solution to the right optimization problem, and vice versa.
2. In general, converting between these two forms is not easy: it’s not always easy to determine what B should be for a particular λ and vice versa without just running the optimization.
3. As a result of the above two considerations, we **almost always deal exclusively with the left formulation**. This is because unconstrained optimization is easier than constrained optimization. The right formulation, however, can lead to better intuition (see below).

Three of the most common instantiations of $t(\mathbf{w})$ are:

1. **L2 regularization.** Here, $t(\mathbf{w}) = \|\mathbf{w}\|_2^2 = \sum_{i=1}^d w_i^2$. Intuitively, this encourages us to not rely overmuch on any particular entry of $\mathbf{w}$, and encourages us to use all features roughly evenly.
2. **L1 regularization.** Here, $t(\mathbf{w}) = \|\mathbf{w}\|_1 = \sum_{i=1}^d |w_i|$. Intuitively, this encourages us to not use some features at all – the higher you set λ the more values of $\mathbf{w}$ that will get set to zero. In other words, $\mathbf{w}$ becomes **sparse**. See the figure below.

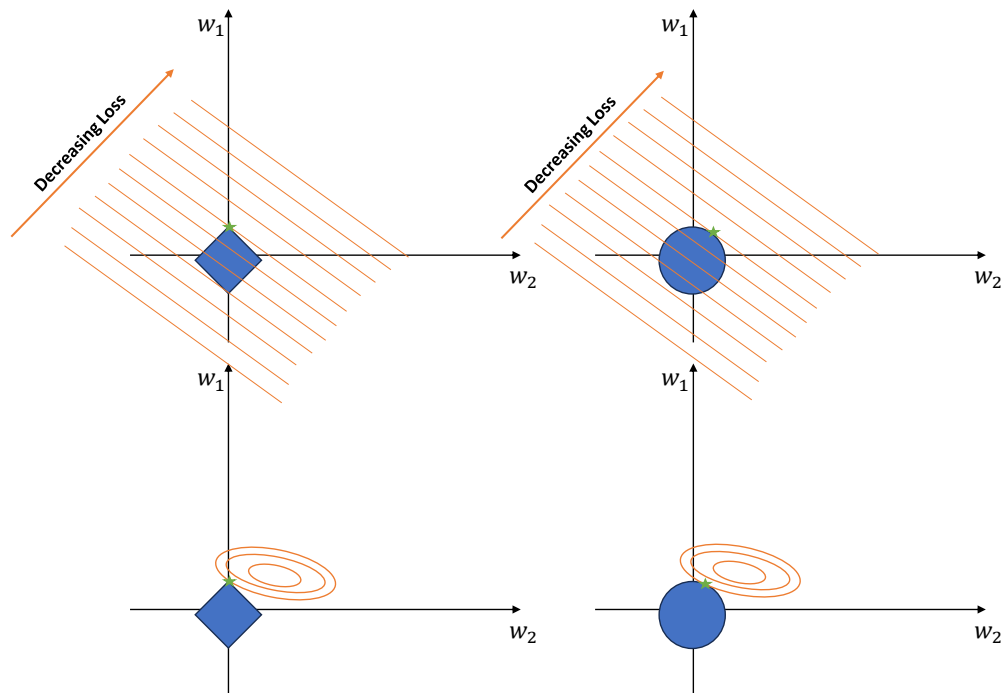


Figure 2: A visual guide to L1 versus L2 regularization. Imagine I am learning a two dimensional $\mathbf{w}$ vector. The axes in the plots are the entries in that vector. The diamonds on the left correspond to L1 regularization constraints, and the circles on the right correspond to L2 regularization constraints. **Top:** in the extremely simple setting where my loss just decreases in some direction, the orange lines represent the “contours” of the loss, where any setting of $\mathbf{w}$ on the same line achieves the same loss. Since the loss is decreasing in that direction, we’ll always choose a $\mathbf{w}$ on the boundary of the constraint set. Unless the lines are parallel to the L1 diamond, we’ll always end up at a corner, corresponding to one weight being zero. **Bottom:** In the case of a quadratic loss, it’s a little harder to reason through, but it turns out again that *most* arbitrary quadratics will intersect the L1 diamond at a corner.

3. **Elastic Net.** Here, we use $\lambda_1 \|\mathbf{w}\|_2^2 + \lambda_2 \|\mathbf{w}\|_1$ as a regularizer. This is a combination of L1 and L2 regularization, and has some “best of both worlds” properties, where we both want some features to be not used at all, and the features that are used to be used roughly evenly.

Ridge regression. **Ridge regression** is a special case of linear regression where we add an L2 regularizer:

$$\mathcal{L}(\mathbf{w}) = \frac{1}{n} \sum_{i=1}^n (\mathbf{w}^\top \mathbf{x}_i - y_i)^2 + \lambda \|\mathbf{w}\|_2^2$$

If you repeat the steps we did for linear regression, you’ll find that we can again compute the optimal $\mathbf{w}^*$ in closed form:

$$\mathbf{w}_{\text{ridge}}^* = (\mathbf{X}^\top \mathbf{X} + \lambda n I)^{-1} \mathbf{X}^\top \mathbf{y}$$

Lasso. The **Lasso** is a special case of linear regression where we add an L1 regularizer:

$$\mathcal{L}(\mathbf{w}) = \frac{1}{n} \sum_{i=1}^n (\mathbf{w}^\top \mathbf{x}_i - y_i)^2 + \lambda \|\mathbf{w}\|_1$$

Unlike linear regression and ridge regression, the lasso has no closed form solution, and numerical optimization (like gradient descent) is actually needed!